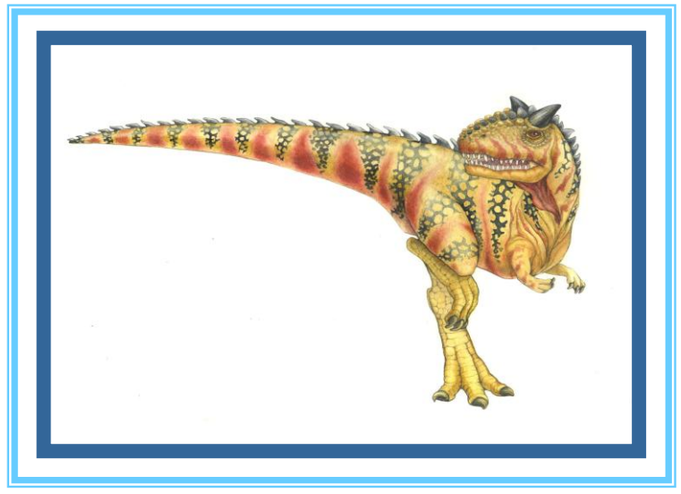
5. Bölüm: İş Sıralama (CPU Scheduling)





5. Bölüm: İş Sıralama (CPU Zamanlama / Planlama / Çizelgeleme)

- Temel Kavramlar
- Sıralama Kriterleri
- İş Sıralama Algoritmaları
- İş Parçacığı Sıralama
- Çoklu-işlemci Sıralama
- İşletim Sistemleri Örnekleri
- Algoritma Değerlendirme





Hedefler

- Çeşitli CPU iş sıralama algoritmalarını tanımlamak
- CPU iş sıralama algoritmalarını değerlendirmek
- Çok işlemcili ve çok çekirdekli iş sıralama ile ilgili sorunları açıklamak
- Çeşitli gerçek zamanlı iş sıralama algoritmalarını tanımlamak
- Windows, Linux ve Solaris işletim sistemlerinde kullanılan iş sıralama algoritmalarını açıklamak
- CPU iş sıralama algoritmalarını değerlendirmek için modelleme ve simülasyon yönteminden faydalanmak
- Belirli bir sistem için CPU İş sıralama algoritma seçim kriterlerini değerlendirmek





İş Parçacığı Sıralama

- Kullanıcı-seviyeli ve çekirdek-seviyeli iş parçacıkları arasında ayrım
- İş parçacıkları desteği varsa prosesler değil iş parçacıkları sıralanır
- Çoktan- teke ve çoktan-çoka modelleri, iş parçacığı kütüphanesi LWP üzerinde çalışmak için kullanıcı seviyeli iş parçacıklarını sıralar
  - İş sıralama yarışı proses içinde olduğu için **proses çekişme kapsamı (process-contention scope-PCS)** olarak bilinir
  - Programcı tarafından öncelik kümesi yardımıyla yapılır
- Mevcut CPU üzerinde sıralanan çekirdek iş parçacığı **sistem çekişme kapsamı (system-contention scope-SCS)** dır– sistemdeki tüm iş parçacıkları arasında yarış





Pthread İş Sıralama

- API iş parçacığının oluşturulması sırasında PCS veya SCS olup olmamasını belirtmeyi olanaklı kılar
  - PTHREAD_SCOPE_PROCESS PCS ile iş parçacıklarını sıralar
  - PTHREAD_SCOPE_SYSTEM SCS ile iş parçacıklarını sıralar
- OS tarafından sınırlandırılabilir – Linux ve Mac OS X sadece PTHREAD_SCOPE_SYSTEM i kullanır





Pthread Sıralama API

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
   int i, scope;
   pthread_t tid[NUM THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
         printf("PTHREAD_SCOPE_PROCESS");
      else if (scope == PTHREAD_SCOPE_SYSTEM)
         printf("PTHREAD_SCOPE_SYSTEM");
      else
         fprintf(stderr, "Illegal scope value.\n");
   }
```





Pthread Sıralama API

```
/* set the scheduling algorithm to PCS or SCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
   pthread_create(&tid[i],&attr,runner,NULL);
/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++)
   pthread_join(tid[i], NULL);
}
/* Each thread will begin control in this function */
void *runner(void *param)
{
   /* do some work ... */
   pthread_exit(0);
}
```





Çok İşlemcili İş Sıralama

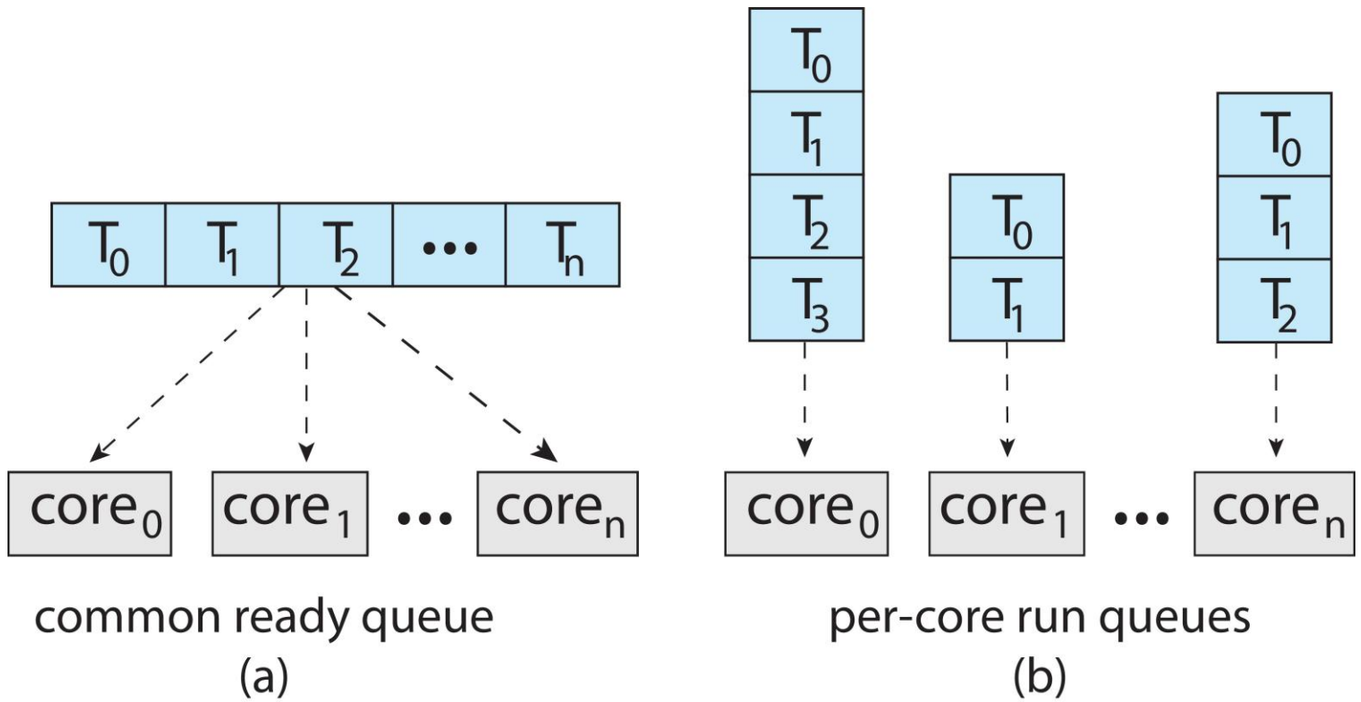
- Birden fazla CPU varsa iş sıralama daha karmaşık olur.
- Çok işlemci aşağıdaki mimarilerden biridir:
  - Çok çekirdekli CPUlar
  - Çoklu İş Parçacıklı çekirdekler
  - NUMA sistemleri
  - Heterojen çok işlemcili sistemler

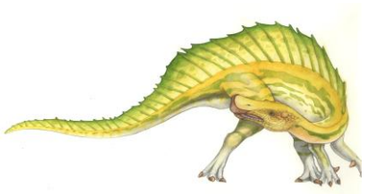




Çok İşlemcili Sıralama

- Simetrik çoklu işlemci (SMP), her işlemcinin kendi kendini sıraladığı bir sistemdir.
- Tüm iş parçacıkları ortak bir hazır kuyruğunda olabilir (a)
- Her işlemcinin kendi özel iş parçacığı kuyruğu olabilir (b)

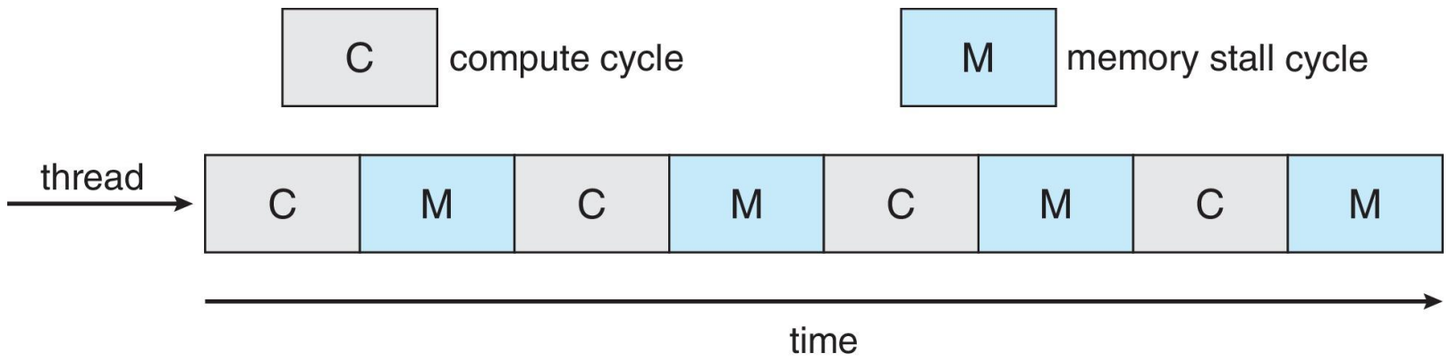






Çok Çekirdekli İşlemciler

- Son eğilim birden fazla işlemci çekirdeğini tek bir fiziksel çip üzerine yerleştirmektir
- Daha hızlı ve daha az güç harcar
- Çekirdek başına birden fazla iş parçacığı ortaya çıkıyor
  - Bellekten veri alınırken bir başka iş parçacığı üzerinde devam etmek için ara verilmesinin avantajını kullanır

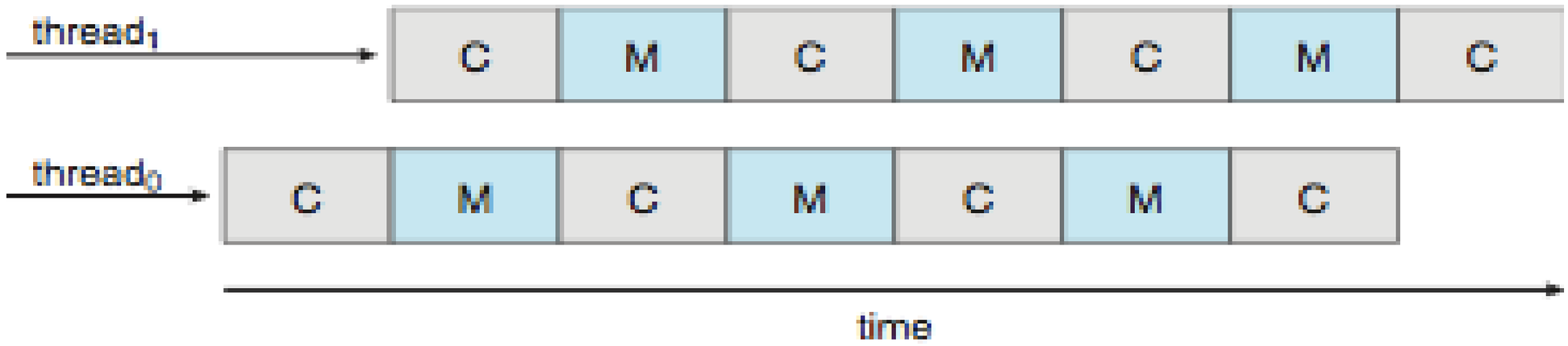






Çoklu İş Parçacıklı Çok Çekirdekli Sistem

- Her çekirdek > 1 donanım iş parçacığına sahiptir.
- Bir iş parçacığının bellek okuma işlemi varsa, başka bir iş parçacığına geçin!

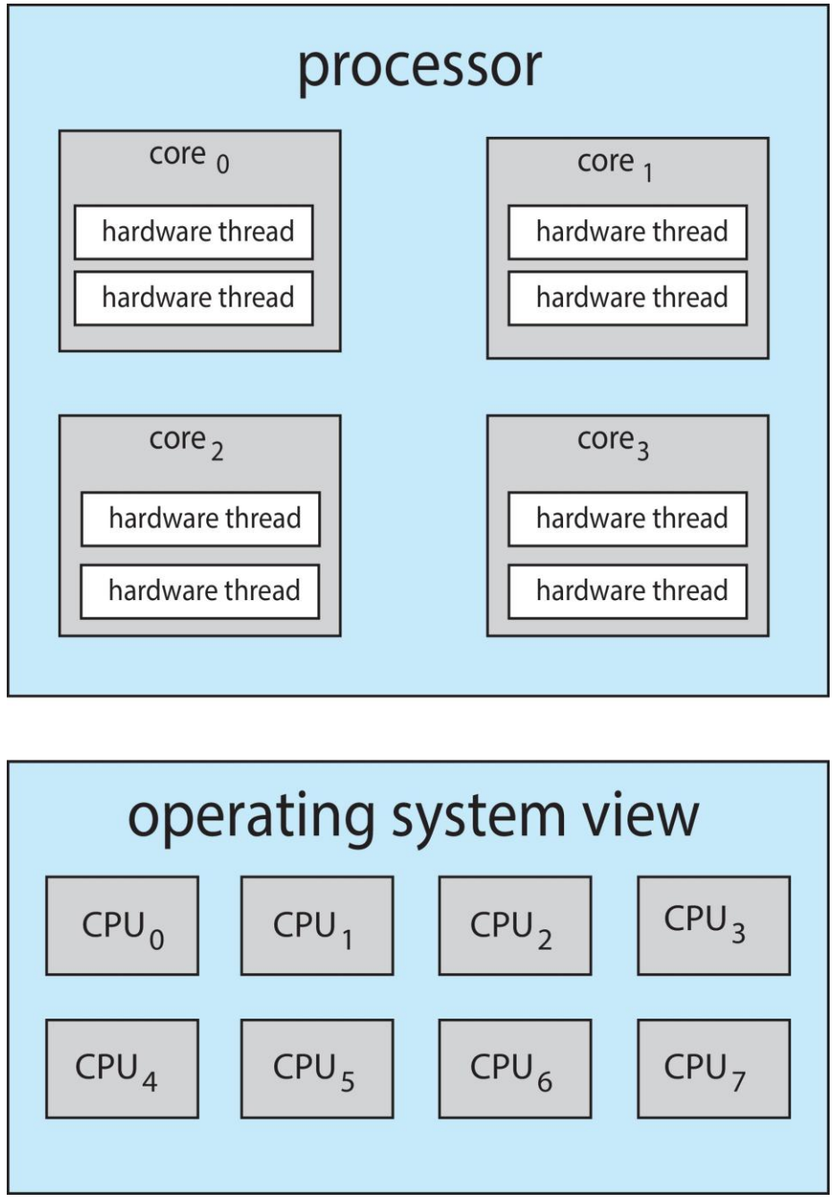




Çoklu İş Parçacıklı Çok Çekirdekli Sistem

- Çip çoklu iş parçacığı (CMT), her çekirdeğe birden çok donanım iş parçacığı atar. (Intel **hyperthreading** olarak adlandırır)

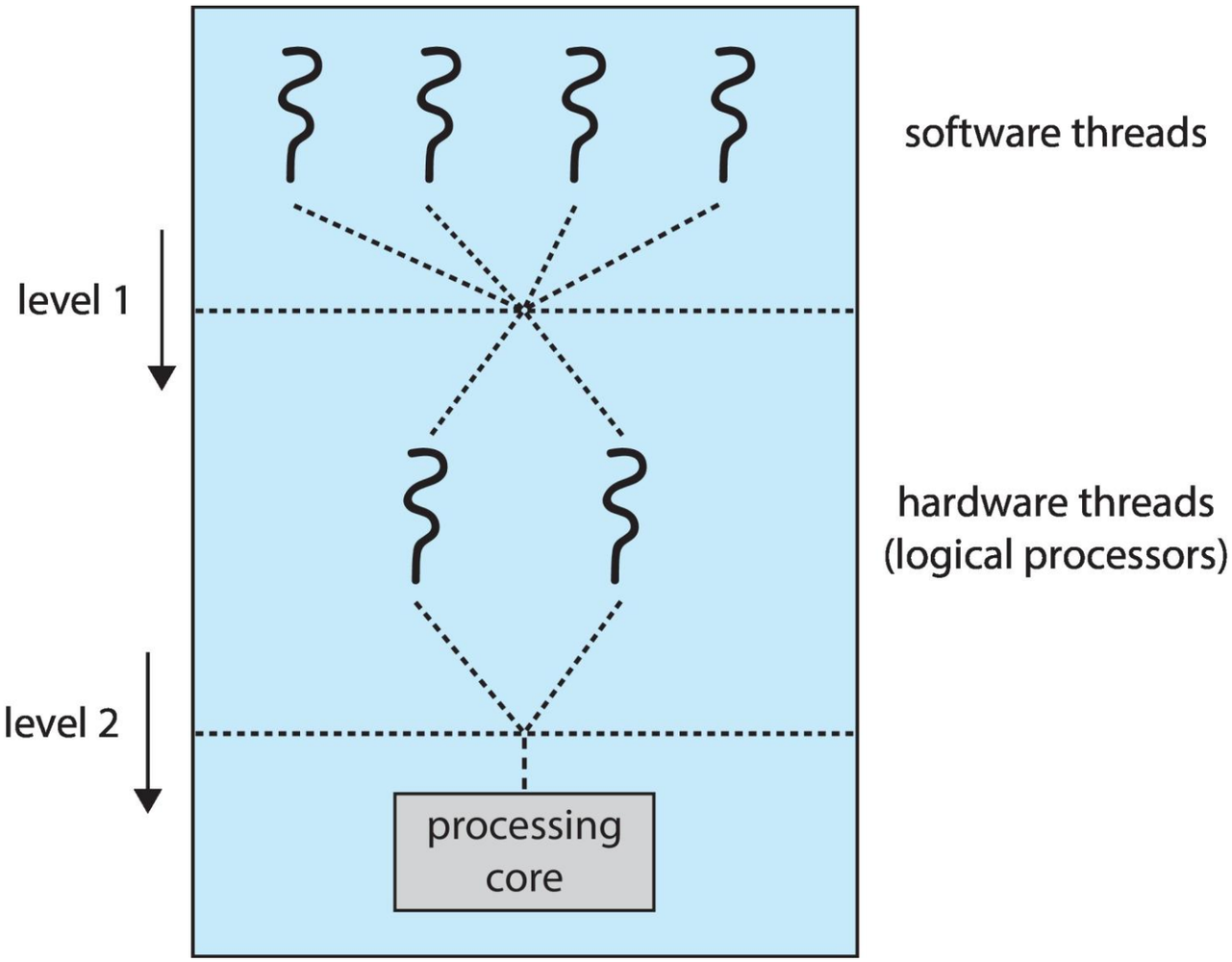
- Çekirdek başına 2 donanım iş parçacığına sahip dört çekirdekli bir sistemde, işletim sistemi 8 mantıksal işlemci görür.





Çoklu İş Parçacıklı Çok Çekirdekli Sistem

- İki sıralama düzeyi:
  1. Işletim sistemi mantiksal CPU'da hangi yazılım iş parçacığının çalıştırılacağına karar verir
  2. Her çekirdek, fiziksel çekirdekte hangi donanım iş parçacığının çalıştırılacağına karar verir







Multiple-Processor Scheduling – Load Balancing

- SMP ise, verimlilik için tüm CPU'ları yüklü tutmanız gerekir Yük dengeleme, iş yükünü eşit şekilde dağıtılmış tutma girişimleri İtme geçişi - periyodik görev her işlemciye yük bindirme sağlar ve bulunursa görevi aşırı yüklenmiş CPU'dan diğer CPU'lara iter Çekme geçişi - boşta kalan işlemciler bekleme görevini meşgul işlemciden çeker



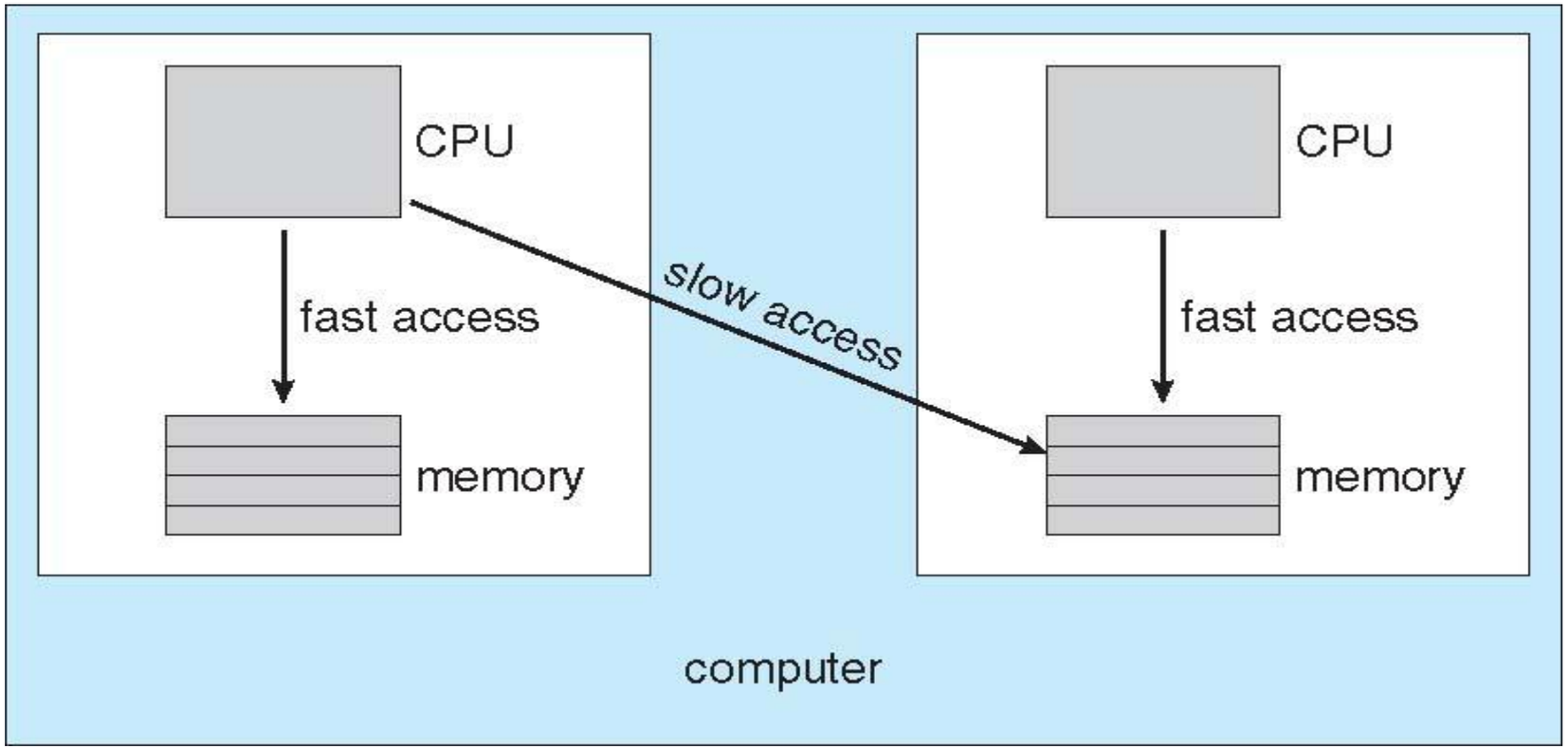


Multiple-Processor Scheduling – Processor Affinity

- When a thread has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
- We refer to this as a thread having affinity for a processor (i.e., “processor affinity”)
- Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of.
- **Soft affinity** – the operating system attempts to keep a thread running on the same processor, but no guarantees.
- **Hard affinity** – allows a process to specify a set of processors it may run on.



NUMA ve CPU İş Sıralama



Bellek yerleşim algoritmalarının da CPU ilişkisini(affinity) göz önünde bulundurduğuna dikkat ediniz



Sanallaştırma ve İş Sıralama

- Sanallaştırma yazılımı birden fazla konuğu CPU üzerine programlar
- Her bir konuk kendi iş sıralama işlemini yapar
  - CPU' ya sahip olup olmadığını bilmeyerek
  - Düşük bir cevap zamanına neden olabilir
  - Konukların güncel zamanları etkilenebilir
- Konukların iyi iş sıralama algoritması çabalarını telafi edebilir





İşletim Sistemi Örnekleri

- Solaris iş sıralama
- Windows XP iş sıralama
- Linux iş sıralama





Solaris

- Öncelik tabanlı iş sıralama
- Altı adet sınıf mevcuttur
  - Zaman paylaşımı (varsayılan)
  - İnteraktif
  - Gerçek zamanlı
  - Sistem
  - Açık paylaşım
  - Sabit öncelik
- Belirli bir iş parçacığı herhangi bir zamanda sadece bir sınıfta olabilir
- Her bir sınıf kendi iş sıralama yaklaşımına sahiptir
- Zaman paylaşımı çok seviyeli geri beslemeli kuyruktur
  - Sistem yöneticisi tarafından yüklenebilir tablo konfigüre edilebilir





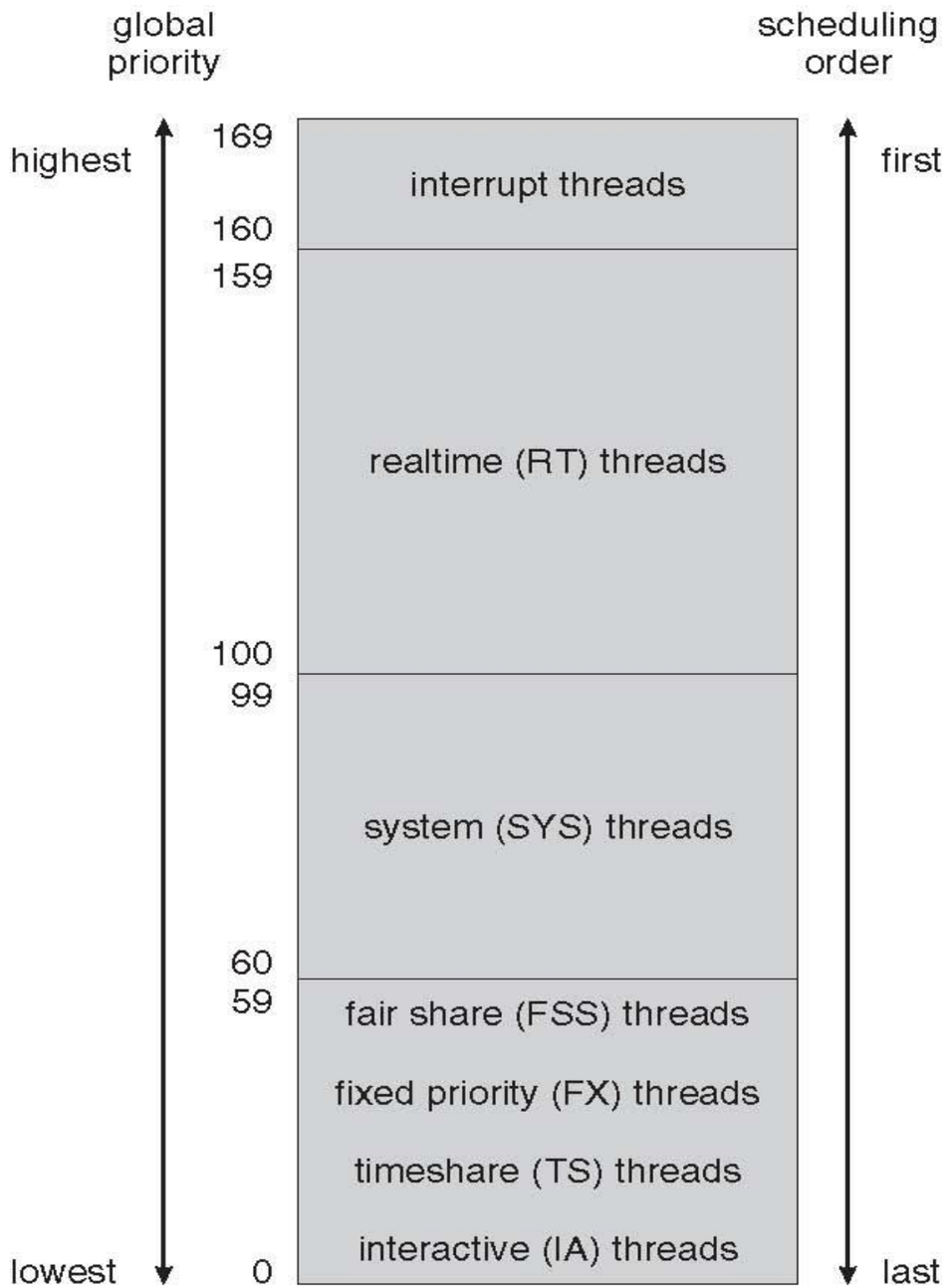
Solaris Görevlendirme Tablosu

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59





Solaris İş Sıralama







Solaris İş Sıralama(devam)

- İş sıralayıcı sınıfa özgü öncelikleri iş parçacığı global önceliğine dönüştürür
  - En yüksek öncelikli iş parçacığı bir sonra çalışır
  - (1) bloke olana kadar çalışır, (2) zaman periyodu kullanılır, (3) daha yüksek öncelikli iş parçacığı tarafından kesilir
  - Aynı öncelikli birden fazla iş parçacığı RR'ye göre çalışır





Windows İş Sıralama

- Windows öncelik tabanlı kesintili bir iş sıralama yaklaşımını kullanır
- En yüksek öncelikli iş parçacığı bir sonra çalışır
- *Görevlendirici (Dispatcher) iş sıralayıcıdır*
- İş parçacıkları (1) bloke olana, (2) zaman periyodu bitimini, (3) daha yüksek öncelikli bir iş parçacığı tarafından kesilene dek çalışır
- Gerçek zamanlı iş parçacıkları olmayanları kesebilir
- 32-seviyeli öncelik düzeni
- **Değişken sınıf** 1-15, **gerçek zamanlı sınıf** 16-31
- 0 önceliği bellek yönetimi iş parçacığıdır
- Her öncelik için kuyruk vardır
- Eğer çalışabilir bir iş parçacığı yoksa, **boşta iş parçacığı** çalışır





Windows Öncelik Sınıflar

- Win32 API prosesin ait olduğu öncelik sınıflarını tanımlar
  - REALTIME_PRIORITY_CLASS, HIGH_PRIORITY_CLASS, ABOVE_NORMAL_PRIORITY_CLASS,NORMAL_PRIORITY_CLASS, BELOW_NORMAL_PRIORITY_CLASS, IDLE_PRIORITY_CLASS
  - REALTIME harici tümü değişkendir
- Belirli bir öncelik sınıfındaki bir iş parçacığı bağıl bir önceliğe sahiptir
  - TIME_CRITICAL, HIGHEST, ABOVE_NORMAL, NORMAL, BELOW_NORMAL, LOWEST, IDLE
- Öncelik sınıfı ve bağıl öncelik nümerik önceliği oluşturur
- Taban önceliği NORMAL dir
- Eğer kuantum zamanı aşarsa, öncelik tabanın altına inmeyecek şekilde düşürülür
- Eğer bekleme olursa, ne için beklediğine göre öncelik artırılır
- En öndeki pencereye 3x öncelik verilir





Windows XP Öncelikleri

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1





Linux İş Sıralama

- Sabit dereceli *O*(1) iş sıralama zamanı
- Kesintili öncelik tabanlı
- İki adet öncelik aralığı: zaman paylaşımlı ve gerçek zamanlı
- **Gerçek zamanlı** aralık 0 dan 99 a dır
- Daha yüksek önceliği gösteren nümerik düşük değerlere sahip global öncelik
- Daha yüksek öncelik daha büyük q ya neden olur
- Zaman aralığında kalan zaman süresi kadar görev çalışabilir (**active**)
- Eğer zaman kalmamışsa (**expired**), diğer görevler kendi zaman aralıklarını kullanana kadar çalışamaz
- Tüm çalışabilir görevler CPU başına bir adet çalışır kuyruğunda tutulur
  - İki adet öncelikli dizi (active, expired)
  - Görevler önceliğe göre sıralanır
  - Aktif görev kalmadığında diziler yer değişir





Linux İş Sıralama (devam)

- Gerçek zaman iş sıralama POSIX e göre yapılır
  - Gerçek zamanlı görevler statik önceliklere sahiptir
- Tüm diğer görevler nice değerine göre artı eksi 5 olarak dinamiktir.
  - Görevin interaktivitesi artı veya eksi olmasını belirler
    - Daha interaktif -> daha eksi
  - Görev zaman aşımına uğradığında öncelik yeniden hesaplanır
  - Bu karşılıklı değişilen diziler ayarlı öncelikleri uygular





Öncelikler ve Zaman Süresi

numeric priority	relative priority		time quantum
0	highest	real-time tasks	200 ms
•			
•			
•			
99			
100		other tasks	
•			
•			
•			
140	lowest		10 ms





Önceliklere Göre Sıralanmış Görev Listesi







Java Thread Scheduling

- JVM Uses a Preemptive, Priority-Based Scheduling Algorithm
- FIFO Queue is Used if There Are Multiple Threads With the Same Priority





Java Thread Scheduling (Cont.)

JVM Schedules a Thread to Run When:

1. The Currently Running Thread Exits the Runnable State
2. A Higher Priority Thread Enters the Runnable State

* Note – the JVM Does Not Specify Whether Threads are Time-Sliced or Not





Time-Slicing

Since the JVM Doesn't Ensure Time-Slicing, the yield() Method
May Be Used:

```
while (true) {
    // perform CPU-intensive task
    . . .
    Thread.yield();
}
```

This Yields Control to Another Thread of Equal Priority





Thread Priorities

Priority	**Comment**
Thread.MIN_PRIORITY	Minimum Thread Priority
Thread.MAX_PRIORITY	Maximum Thread Priority
Thread.NORM_PRIORITY	Default Thread Priority

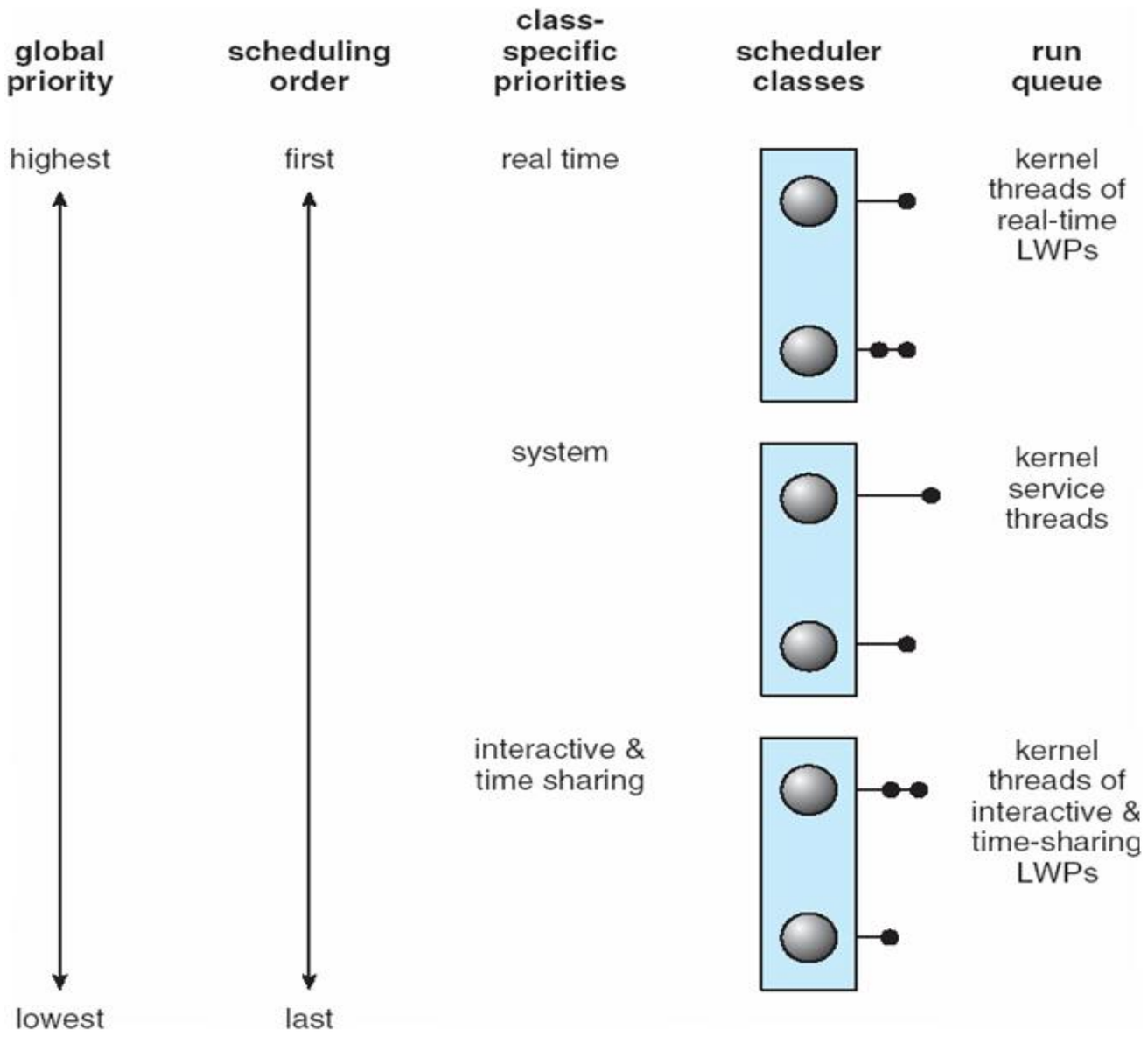
Priorities May Be Set Using setPriority() method:

```
setPriority(Thread.NORM_PRIORITY + 2);
```





Solaris 2 Scheduling







Algoritma Değerlendirme

- Bir işletim sistemi için CPU iş sıralama algoritması nasıl seçilir ?
- Kriterleri belirle ve daha sonra algoritmaları değerlendir
- Deterministik modelleme
  - Bir tür analitik değerlendirme
  - Belirli bir iş yükünü alır ve o iş yükü için herbir algoritmanın performansını hesaplar





Kuyruk Modelleri

- Proseslerin varışını, CPU ve I/O patlamalarını olasılıksal olarak tanımlar
  - Genelde üstel ve ortalama ile tanımlanır
  - Ortalama çıkış(throughput), kullanım oranı, bekleme zamanını hesaplar
- Bilgisayar sistemini herbiri bekleyen prosesler kuyruğuna sahip sunucular ağı olarak tanımlar
  - Varış oranı ve hizmet oranı bilinir
  - Kullanım oranını, ortalama kuyruk boyutunu ve ortalama bekleme zamanını hesaplar





Little Formülü

- *n* = ortalama kuyruk boyutu
- *W* = kuyruktaki ortalama bekleme zamanı
- *λ* = ortalama kuyruğa varış oranı
- Little'ın kuralı – kararlı durumda, kuyruğu terk eden prosesler kuyruğa varanlarla eşit olmalıdır, bu nedenle
- $n = \lambda \times W$
  - Herhangi bir iş sıralama algoritması ve geliş dağılımı için geçerlidir
- Mesela, eğer saniyede ortalama 7 proses varıyorsa ve normalde kuyrukta 14 proses varsa prosesler için ortalama bekleme zamanı = 2 saniye olur.





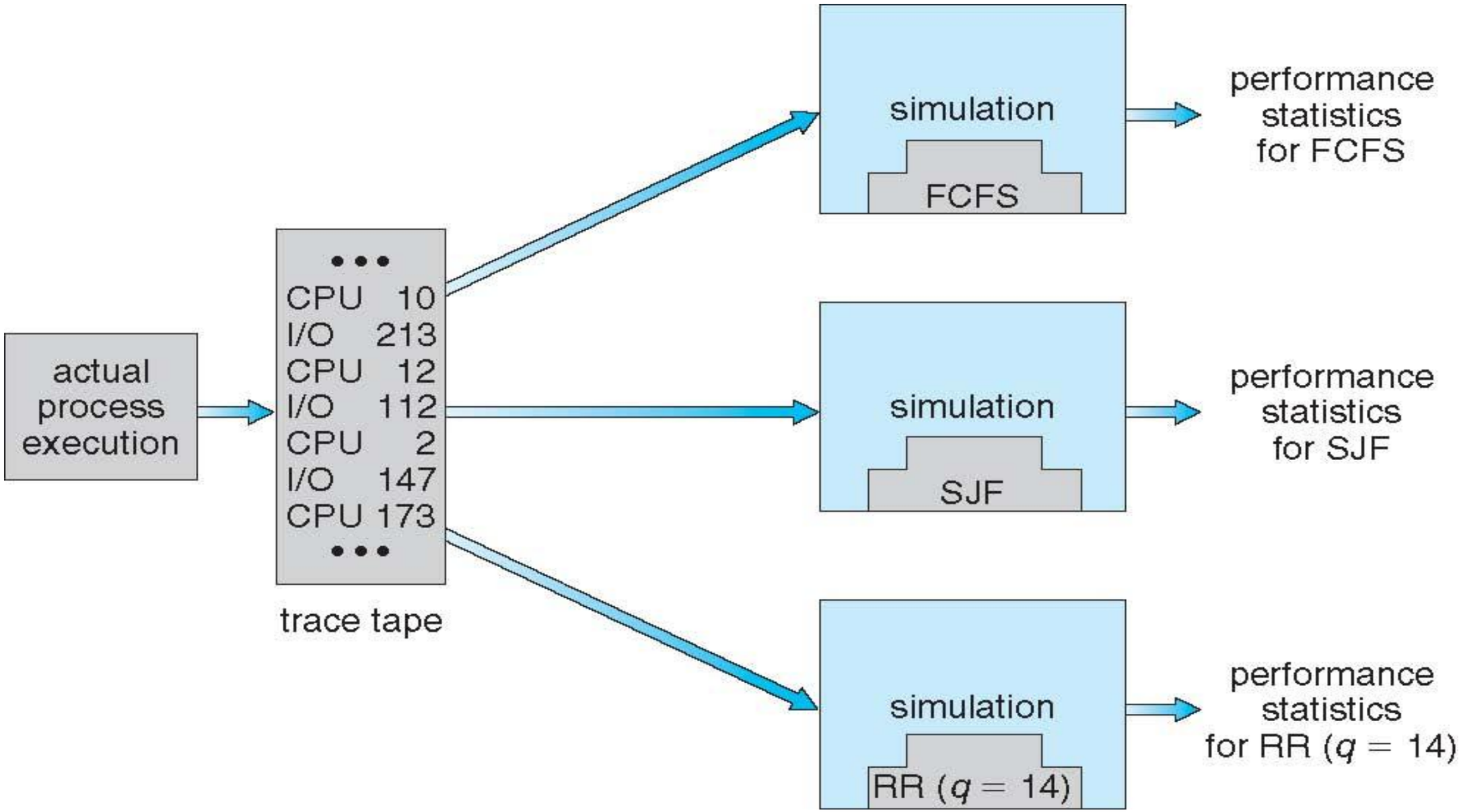
Simülasyonlar

- Kuyruk modelleri sınırlıdır
- **Simülasyonlar** daha doğru sonuç verir
  - Bilgisayar sistemi modeli
  - Saat değişkendir
  - Algoritma performansını gösteren istatistikler üret
  - Simülasyonu süren veri :
    - Rasgele sayı üreteci
    - Matematiksel dağılımlar
    - Gerçek sistemlerdeki gerçek olayların sırasını tutan kayıtlar





CPU İş Sıralayıcıların Simülasyon ile Değerlendirilmesi







Uygulama

- Simülasyonlar da sınırlı doğruluğa sahiptir
- Sadece yeni iş sıralayıcıyı uyarla ve gerçek sistemlerde test et
  - Yüksek maliyet, yüksek risk
  - Çevre değişebilir
- Çok esnek iş sıralayıcılar mekana veya sisteme göre değiştirilebilir
- Veya API'ler öncelikleri değiştirmek maksadıyla geliştirilebilir
- Ancak çevre değişkendir



5. Bölümün Sonu

